Workload-Forecasting Driven Proactive Pod Scheduling for Kubernetes Clusters using LSTM-XGBoost Ensemble and Multi-Objective Node Ranking

1.PROBLEM DEFINITION:

Kubernetes' default Horizontal Pod Autoscaler (HPA) and default scheduler are fundamentally reactive: scaling and placement decisions are triggered only after CPU or memory utilization crosses a threshold, by which time a request surge has already begun to degrade response time. Real-world web and microservice workloads are highly time-varying and exhibit daily, weekly and bursty patterns, and single-metric, threshold-based logic cannot anticipate these patterns or account for multiple competing objectives (CPU headroom, memory headroom, latency and load balance across nodes) at the same time.

This project addresses the problem of proactively scheduling pods in a Kubernetes cluster by (i) forecasting near-future workload/resource demand using a hybrid LSTM-XGBoost ensemble that captures both long-term temporal dependencies and non-linear feature interactions, and (ii) using this forecast to rank candidate nodes with a multi-objective scoring function before pod placement, so that scaling and placement decisions are made ahead of demand rather than after SLA violations have already occurred.

2.ABSTRACT:

Kubernetes has become the standard platform for orchestrating containerized microservices, yet its built-in autoscaling and scheduling mechanisms rely on reactive, single-metric thresholds that often fail to anticipate rapidly changing workloads. This leads to either resource over-provisioning or Service Level Objective (SLO) violations during sudden traffic surges. This project proposes a proactive pod scheduling framework that combines a Long Short-Term Memory (LSTM) network with an XGBoost regressor in an ensemble to forecast short-term CPU, memory and request-rate demand from historical and real-time Prometheus metrics. The fused forecast is then fed into a multi-objective node ranking engine that scores candidate worker nodes on predicted resource availability, latency impact and cluster-wide load balance,

and forwards the top-ranked node to a custom scheduler extension for pod placement. The objective is to shift Kubernetes scaling and placement from a reactive, single-metric process to a forecast-driven, multi-criteria one. The proposed system is evaluated against the default HPA and single-model (LSTM-only) baselines using metrics such as prediction error (MSE/RMSE), average response time, SLO violation percentage and resource utilization. The expected outcome is a scheduler extension that reduces SLO violations and improves resource efficiency compared to reactive and single-model forecasting approaches, while remaining deployable as a lightweight add-on to existing Kubernetes clusters.

3. LITERATURE REVIEW:

Four recent papers were reviewed to establish the technical foundation and identify the gap this project fills.

Amirullah & Saikhu (IAICT 2025) – CPU Usage Forecasting for Load Balancing in Kubernetes Using LSTM: Compared ARIMA, GRU and LSTM for CPU usage forecasting using Beta-distribution-based synthetic traffic generated with k6. LSTM achieved the lowest MSE (0.00001053) and RMSE (0.00324451). Relevant because it validates LSTM as a strong single-model baseline and the synthetic-traffic-simulation methodology, but it stops at forecasting only and does not integrate the prediction into an actual scheduling/placement decision.

Lipari, Proietti Mattia & Beraldi (IPDPSW 2025) – Dynamic and Forecast-based Containers Autoscaling with Reinforcement Learning: Combined an LSTM workload predictor with a Double DQN reinforcement-learning agent to decide replica counts, reducing resource consumption by up to 10% and improving SLO compliance versus HPA. Relevant for its predictor-plus-decision-engine architecture and its deployment/evaluation pipeline on GKE, which this project adapts by replacing the RL replica-decision step with a multi-objective node-ranking step for placement.

Toka, Dobreff, Fodor & Sonkoly (CCGRID 2020) – Adaptive AI-based Auto-scaling for Kubernetes (HPA+): Proposed HPA+, which races AR, HTM and LSTM forecasters against each other every minute and falls back to default HPA when accuracy is poor, plus an 'excess'

parameter to trade off over-provisioning against SLA violations. Relevant for the idea of combining multiple forecasting models and for a configurable objective/priority parameter, which motivates this project's LSTM-XGBoost ensemble and multi-objective node score.

Matsiievskyi et al. (SIST 2025) – Application of Neural Networks to Optimize Distributed Computing in Cloud and Edge Environments: Used an LSTM model trained on a synthetic dataset (sinusoidal base load + noise + spikes) to forecast server load for proactive resource allocation, showing good tracking of stable trends but a lag during sudden spikes. Relevant for its synthetic-data generation approach for training/testing and for explicitly identifying sudden-spike prediction lag as an open limitation, which this project addresses by adding XGBoost to the ensemble to better capture non-temporal, feature-driven jumps.

Across all four papers, LSTM is consistently used as the core temporal forecaster, and every paper about which integrates forecasting into the actual Kubernetes control loop, either extends beyond raw forecasting, using it to drive replica counts (paper 2), model switching (paper 3), or scaling policy (paper 1), but none of them combines a second, feature-based model (XGBoost) with LSTM, and none of them turns the forecast into a multi-objective, per-node ranking for placement rather than a single scale-up/down number. This gap — ensemble forecasting plus multi-objective node ranking for proactive pod scheduling — is the specific contribution of this project.

4. EXISTING SYSTEM:

The current, widely deployed Kubernetes system relies on the Horizontal Pod Autoscaler (HPA) and the default kube-scheduler:

- HPA polls a single metric (typically average CPU utilization) at fixed intervals and scales replica count only after the metric crosses a static threshold — a purely reactive decision.
- HPA parameters (target utilization, min/max replicas, scaling tolerance, downscale stabilization window) are configured once and remain static even as workload variability changes over time.

- The default scheduler places new pods based on instantaneous resource requests/limits and simple filtering/scoring rules; it does not consider predicted future load on a node or a combination of multiple objectives such as latency and cluster-wide balance.
- Because scaling is reactive, there is an inherent lag between the onset of a traffic surge and the availability of new pods/nodes to absorb it, causing temporary latency spikes and SLO violations.
- Existing research prototypes (LSTM-only forecasters, RL-based autoscalers, multi-forecaster racing engines such as HPA+) improve on raw HPA but either forecast without acting on placement, or act using a single model/objective, leaving room for a combined ensemble-forecast plus multi-objective placement approach.

5. PROPOSED SYSTEM:

The proposed system replaces the reactive threshold logic with a forecast-driven, multi-objective proactive scheduling pipeline. Its objectives are:

- Forecast near-future CPU, memory and request-rate demand using an ensemble of LSTM (captures long/short-term temporal dependencies) and XGBoost (captures non-linear, feature-based patterns such as time-of-day and recent rate-of-change), combined through a weighted or stacked fusion layer.
- Continuously collect real-time cluster and workload metrics from Prometheus so that the forecast is retrained/updated on live data rather than only on offline historical data.
- Convert the fused forecast into a per-node Multi-Objective Node Ranking score that jointly considers predicted CPU/memory headroom, expected latency impact and cluster-wide load balance, instead of a single-metric threshold.
- Feed the top-ranked node(s) into a custom Kubernetes scheduler extension so that new/rescheduled pods are proactively placed on the node best suited to the predicted near-future load, ahead of the demand actually arriving.

- Evaluate the system against the default HPA and single-model (LSTM-only) baselines using MSE/RMSE of the forecast, average response time, percentage of SLO violations and average resource (CPU/replica) usage.

This design is feasible because it builds directly on components already validated in the reviewed literature — LSTM-based forecasting (Papers 1, 2 and 4) and metric collection via Prometheus with Kubernetes custom-metrics APIs (Papers 1–4) — while its novelty lies in the LSTM-XGBoost ensemble fusion and the multi-objective node-ranking step that turns a forecast into an actionable, multi-criteria placement decision.

6. ARCHITECTURE DIAGRAM:

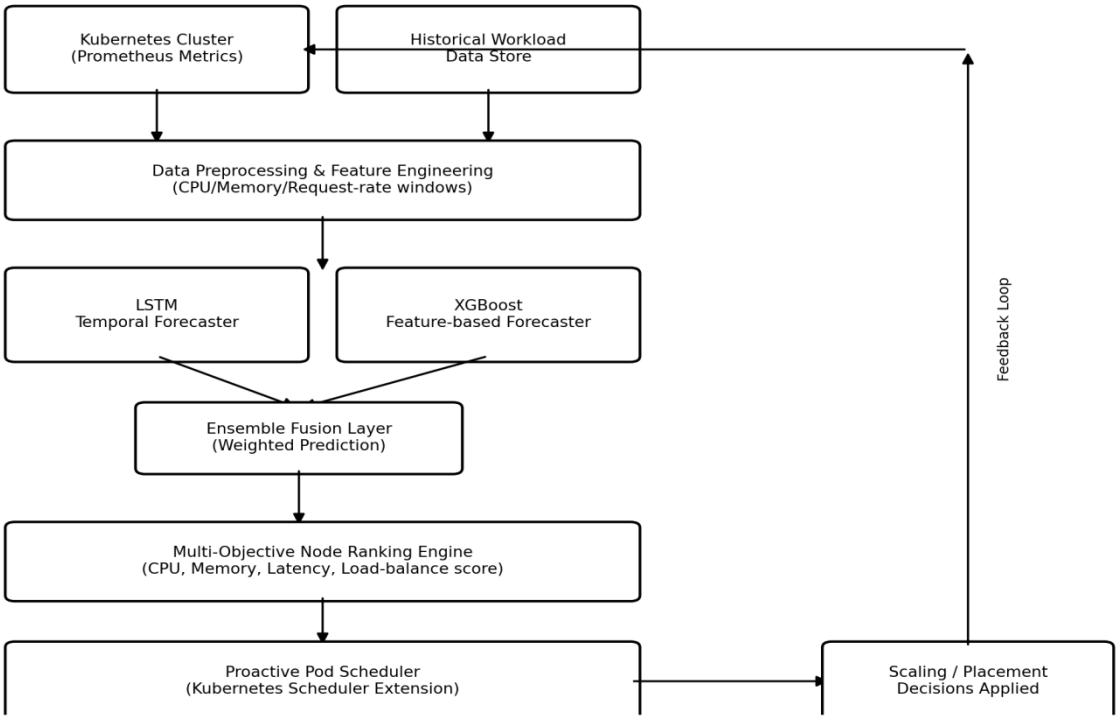

Figure 1. End-to-end architecture: live cluster metrics and historical workload data are preprocessed and fed to the LSTM and XGBoost forecasters in parallel; their outputs are fused into a single prediction, which drives the Multi-Objective Node Ranking engine; the highest-ranked node is passed to the Proactive Pod Scheduler, which applies the placement/scaling decision to the cluster, closing the feedback loop back to metric collection.

7. MODULE SPECIFICATION:

1.Metrics Collection Module:

Continuously scrapes CPU, memory, request-rate and latency metrics from Prometheus/Kubernetes Metrics API and stores them as a time-series feature store for training and inference.

2. Data Preprocessing Module:

Cleans, normalizes and windows the raw time-series into fixed-length sequences (for LSTM) and engineered tabular features such as lag values and time-of-day (for XGBoost).

3. LSTM Forecasting Module:

A recurrent network that learns long- and short-term temporal dependencies in the workload series and outputs a short-horizon demand forecast.

4. XGBoost Forecasting Module:

A gradient-boosted tree regressor trained on engineered features to capture non-linear, non-sequential patterns and sudden-change signals that complement the LSTM output.

5. Multi-Objective Node Ranking Module:

Scores every candidate worker node using the fused forecast against multiple objectives predicted CPU/memory headroom, estimated latency and cluster load balance and ranks nodes for placement.

6. Proactive Pod Scheduler Module:

A Kubernetes scheduler extension/plugin that consumes the node ranking and binds new or rescheduled pods to the top-ranked node ahead of demand.

7. Monitoring & Evaluation Module:

Dashboards and logging (e.g., Grafana) that track forecast accuracy (MSE/RMSE), response time, SLO violations and resource usage for comparison against baseline HPA.

8. TECHNOLOGY STACK

Layer	Tools / Technologies
Container Orchestration	Kubernetes, Docker
Monitoring & Metrics	Prometheus, Grafana, Kubernetes Metrics API
Forecasting Models	Python, TensorFlow/Keras (LSTM), XGBoost
Data Processing	NumPy, Pandas, scikit-learn
Load Simulation	k6 / synthetic traffic generator scripts
Scheduler Extension	Kubernetes Scheduler Framework (Go / Python client), Custom Metrics Adapter
Deployment Environment	Google Kubernetes Engine (GKE) / Minikube (for local testing)
Version Control & Docs	Git/GitHub, Jupyter Notebook, Matplotlib

9. DOCUMENT SUBMISSION:

Guide Name: Mr.P.Manikanda Prabu AP/CSE

Signature: _____

Date: _____